

映射：从视觉骨架到舵机

—— 视觉识别的灵巧手控制中，"3D 手部骨架 → 舵机" 到底怎么映射

核心澄清：根本不需要算"功率"

常见误解是以为要计算"舵机输出多少功率/力矩"。其实整条链路输出的是"目标位置(角度)"，不是功率。

Feetech SCS0009 这类总线舵机是**位置闭环**的：你只告诉它"转到 X 度"，它**内部自带 PID 控制器**，自己决定用多大电流/力矩去到达并保持那个角度。所以你**永远不用计算功率/力矩**——那是舵机内部干的事。

完整映射链路（从骨架到舵机）



1. **MediaPipe** 输出手部 21 个关键点的 3D 坐标（指尖、指节）。

2. 从骨架里取**指尖的目标位置**（相对手腕/掌心），作为 3D 空间里的目标点。
3. **逆运动学(IK)**：用 **MuJoCo 里的灵巧手模型 + Mink 求解器**，回答"关节转到多少度，仿真里的指尖才能到达这些目标点"，输出 **8 个电机的目标角度**。
4. **控制节点**把这些角度加上每个电机的标定偏移/方向
(`r_hand.toml`)，作为**目标位置**用串口协议发给舵机。舵机内部 PID 自己发力到位。

回答两个具体问题

Q1: 需要机械结构建模吗?

需要，但通常已经做好了。 第③步的 IK 必须有手的机械模型——在这类开源项目里是 **MuJoCo 的 MJCF 文件** (`robot.xml` 等)，描述了每根手指的连杆、关节、并联结构。IK 正是靠这个模型把"指尖位置"换算成"关节角度"。每根手指 2 个舵机的并联机构关系，也建模在里面。**不用自己重新建模，用项目自带的即可。**

Q2: 从识别到舵机输出怎么映射?

- 识别 → **3D 指尖目标点** (MediaPipe)
- 目标点 → **关节角度** (IK, 靠机械模型)
- 关节角度 → **舵机目标位置** (加标定偏移/方向)
- 舵机自己用内部 PID 发力到位 (**这里才涉及"功率/力矩", 但不用你算**)

关键点：位置控制 ≠ 功率控制。上位机只发"目标角度"，力矩由舵机内部闭环自动分

配。

一句话总结：不是"映射到功率"，而是"映射到目标角度"。视觉给出指尖的 3D 位置 → 用 MuJoCo 里的手模型做逆运动学算出每个关节该转多少度 → 把角度当目标位置发给舵机，舵机内部闭环自己发力到位。机械建模是逆运动学这一步需要的，项目里已用 MJCF 提供好了。

对应到具体代码/节点 (Amazing Hand 项目)

- `HandTracking/main.py` —— MediaPipe 出 3D 骨架 / 指尖目标
- `AHSimulation/mj_mink_right.py` —— MuJoCo + Mink 做逆运动学，出关节角
- `AHControl` (Rust) —— 把关节角 + 标定(`r_hand.toml`) 作为目标位置发给舵机